Explicação aprofundada do código

Proposta 10 — Regularização de sinistros. Este documento reproduz o código por blocos e explica cada linha: o que faz, porque está escrito assim e como o explicar numa apresentação. Os resultados mostrados (em fundo amarelo) são o output real da execução do programa.

0. Visão geral

O programa lê uma tabela de 12 sinistros, assinala os que estão **fora de prazo**, calcula o **valor pendente**, resume tudo por ramo e exporta uma lista de **processos prioritários**. Está dividido em dois ficheiros:

- `funcoes_auxiliares.py` — as regras de negócio (nomes dos ramos, limites em dias, função que decide se um sinistro cumpre o prazo).
- **ficheiro principal** — lê os dados, aplica as regras, produz resumos e exporta o CSV.

```
dados_trabalho_10_seguros.csv
        |  pd.read_csv
        v
  DataFrame "df"  --normalizar_ramo-->  ramos com nomes padrão
        |
        +-- valor_pendente  (reclamado se Pendente, senão 0)
        +-- fora_prazo      (usa cumpre_prazo -> limite_regularizacao -> LIMITES)
        |
        +-- groupby("ramo") -> resumo por ramo
        +-- filtro (Pendente OU fora_prazo) + ordenação -> sinistros_prioritarios.csv
```

Conceito-chave: um **DataFrame** é uma tabela (linhas = sinistros, colunas = variáveis). Uma **Series** é uma coluna (ou uma linha) dessa tabela. Quase tudo o que o programa faz é criar colunas novas a partir de colunas existentes e filtrar/agrupar linhas.

Parte A — funcoes_auxiliares.py

A.1 Import e dicionários

```
import pandas as pd

MAPEAMENTO_RAMO = {
    "Automovel": "Automovel",
    "Automóvel": "Automovel",
    "Habitacao": "Habitacao",
    "Habitação": "Habitacao",
    "Viagem": "Viagem",
    "Responsabilidade Civil": "ResponsabilidadeCivil",
    "ResponsabilidadeCivil": "ResponsabilidadeCivil",
}

LIMITES = {
    "Automovel": 25,
    "Habitacao": 25,
    "Viagem": 10,
    "ResponsabilidadeCivil": 35,
}
```

- `import pandas as pd`: carrega a biblioteca pandas e dá-lhe o apelido `pd`. É usada aqui só para `pd.isna()`.
- **Dicionário** (`dict`): coleção de pares *chave: valor*. Procurar por chave é imediato, como numa lista de contactos.
- `MAPEAMENTO_RAMO`: a chave é o texto como pode *aparecer* nos dados e o valor é o nome *padrão*. Por exemplo, `"Automóvel"` (com acento) passa a `"Automovel"`. As entradas em que chave e valor são iguais (como `"Viagem": "Viagem"`) existem para o nome padrão também ser reconhecido.
- `LIMITES`: os prazos de regularização em dias, que vêm do enunciado (25, 25, 10, 35). Estão **fora das funções** porque são dados do problema, não lógica; se o limite mudar, altera-se num único sítio.
- Os nomes em maiúsculas são uma **convenção** para constantes (valores que não mudam durante o programa).

A.2 normalizar_ramo

```
def normalizar_ramo(ramo):
    if pd.isna(ramo):
        return ramo
    valor = str(ramo).strip()
    return MAPEAMENTO_RAMO.get(valor, valor)
```

- `def normalizar_ramo(ramo):` define uma **função** com um parâmetro. Só executa quando for chamada.
- `pd.isna(ramo)`: devolve `True` se o valor for vazio (`NaN`/`None`). Se for, `return ramo` devolve-o sem o tocar: não se pode fazer `.strip()` a um valor em falta.
- `str(ramo)` garante que é texto; `.strip()` remove espaços no início e no fim (`" Viagem "` → `"Viagem"`).
- `MAPEAMENTO_RAMO.get(valor, valor)`: procura `valor` no dicionário. O **segundo argumento** é o que devolve se não encontrar. Com `dict[valor]` um ramo desconhecido daria `KeyError` e o programa pararia; com `.get(valor, valor)` devolve o texto original.

```
normalizar_ramo(" Responsabilidade Civil ")  ->  "ResponsabilidadeCivil"
normalizar_ramo("Automóvel")                 ->  "Automovel"
normalizar_ramo("Moto")                      ->  "Moto"   (desconhecido: fica como está)
normalizar_ramo(float("nan"))                ->  nan      (em falta: fica em falta)
```

A.3 limite_regularizacao

```
def limite_regularizacao(ramo):
    return LIMITES.get(normalizar_ramo(ramo))
```

- Uma linha, mas faz dois passos encadeados: primeiro `normalizar_ramo(ramo)` (limpa o nome) e depois `LIMITES.get(...)` (procura o limite).
- Aqui `.get()` tem **um só argumento**, por isso devolve `None` quando o ramo não existe. Esse `None` é depois tratado em `cumpre_prazo`.
- É a função exigida pelo enunciado (`limite_regularizacao(ramo)`).

```
limite_regularizacao("Habitação")  ->  25
limite_regularizacao("Viagem")     ->  10
limite_regularizacao("Moto")       ->  None
```

A.4 cumpre_prazo — a regra central

```
def cumpre_prazo(ramo, dias_resolucao):
    limite = limite_regularizacao(ramo)
    if limite is None or pd.isna(dias_resolucao):
        return True
    return dias_resolucao <= limite
```

- `limite = limite_regularizacao(ramo)`: obtém o limite em dias para o ramo.
- `if limite is None or pd.isna(dias_resolucao):` trata as duas situações em que **não é possível avaliar**: ramo sem limite (`None`) ou dias em falta. Usa-se `is None` (e não `== None`) por ser a forma correta de testar `None`.
- `return True`: decisão de negócio — **sem informação, não se assinala atraso**. Evita falsos alarmes, mas tem um custo: um ramo mal escrito passaria por 'dentro do prazo'.
- `return dias_resolucao <= limite`: a comparação devolve `True` ou `False`. O `<=` torna o limite **inclusivo**: resolver em exatamente 25 dias em Habitacao ainda cumpre.

ramo	dias	limite	cumpre_prazo	fora_prazo
Habitacao	25	25	True	False
Habitacao	32	25	False	True
Viagem	14	10	False	True
Moto (desconhecido)	99	None	True	False
Viagem	(em falta)	10	True	False

Nota: o pseudocódigo do trabalho descreve exatamente esta função: *limite ← limite_regularizacao(ramo); se limite ou dias em falta, considerar dentro do prazo; cumpre_prazo ← dias ≤ limite; fora_prazo ← NÃO cumpre_prazo*. A última linha (a negação) é feita no ficheiro principal.

Parte B — Ficheiro principal

B.1 Imports e constantes

```
from funcoes_auxiliares import normalizar_ramo, limite_regularizacao, cumpre_prazo
from pathlib import Path
import pandas as pd

COL_ID = "sinistro"
COL_RAMO = "ramo"
COL_CANAL = "canal"
COL_ESTADO = "estado"
COL_RECLAMADO = "valor_reclamado"
COL_REGULARIZADO = "valor_regularizado"
COL_DIAS = "dias_resolucao"
```

- `from funcoes_auxiliares import ...`: traz para este ficheiro as três funções definidas no outro, que têm de estar **na mesma pasta**. Como se importaram os nomes diretamente, usam-se sem prefixo.
- `from pathlib import Path`: `Path` representa caminhos de ficheiros de forma independente do sistema operativo (Windows usa `\`, Linux `/`).
- As constantes `COL_*` guardam os nomes das colunas do CSV. Em vez de escrever `"dias_resolucao"` em dez sítios, escreve-se `COL_DIAS`: uma gralha dá erro imediato e, se o nome mudar, corrige-se numa linha.

B.2 A classe Sinistro

```
class Sinistro:
    """Representa um sinistro e calcula o respetivo valor pendente."""

    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0
```

- Uma **classe** é um molde; uma **instância** (ou objeto) é um sinistro concreto criado a partir desse molde.
- `__init__` é o **construtor**: corre automaticamente ao criar uma instância. `self` é a própria instância; `self.estado = estado` guarda o valor recebido como **atributo** do objeto.
- `valor_pendente(self)` é um **método**: uma função que pertence à classe e acede aos atributos através de `self`.
- A linha do `return` é uma **expressão condicional**: devolve o valor reclamado *se* o estado for `"Pendente"`, *senão* 0. É a mesma regra da coluna `valor_pendente` do DataFrame, mas aplicada a **um** sinistro.

```
s = Sinistro("S002", "Habitacao", "Pendente", 6200)
s.valor_pendente()   ->  6200
t = Sinistro("S001", "Automovel", "Fechado", 3800)
t.valor_pendente()   ->  0
```

B.3 main(): leitura do ficheiro

```
def main():
    # 1. Leitura e qualidade dos dados
    base_dir = Path(__file__).resolve().parent
    csv_file = base_dir / "dados_trabalho_10_seguros.csv"

    if not csv_file.exists():
        raise FileNotFoundError(
            f"Ficheiro de dados não encontrado: {csv_file}\n"
            "Verifique se o CSV está na mesma pasta do script."
        )

    df = pd.read_csv(csv_file)
```

- `__file__` é o caminho do script em execução; `.resolve()` torna-o absoluto e `.parent` dá a pasta onde está. Assim o CSV é encontrado **mesmo que corras o programa a partir de outra pasta**.
- `base_dir / "ficheiro.csv"`: o operador `/` do `Path` junta partes de um caminho (não é divisão).
- `csv_file.exists()` verifica se o ficheiro existe. Se não existir, `raise FileNotFoundError(...)` para o programa com uma mensagem clara em vez de um erro críptico do pandas.
- O texto com `f"..."` é uma **f-string**: o que está entre chaves (`{csv_file}`) é substituído pelo valor da variável.
- `pd.read_csv(csv_file)` lê o CSV para um DataFrame `df`: 12 linhas, 7 colunas. O enunciado exige `Path` + `pd.read_csv()`, que é isto.

B.4 Limpeza e verificação de qualidade

```
    df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
    df[COL_DIAS] = pd.to_numeric(df[COL_DIAS], errors="coerce")
    df[COL_RECLAMADO] = pd.to_numeric(
        df[COL_RECLAMADO], errors="coerce").fillna(0)
    df[COL_REGULARIZADO] = pd.to_numeric(
        df[COL_REGULARIZADO], errors="coerce").fillna(0)

    print(df.dtypes)
    print(df.isna().sum())
    print("Duplicados:", df.duplicated().sum())
    print(df[COL_ESTADO].unique(), df[COL_RAMO].unique())
```

- `df[COL_RAMO]` seleciona a coluna ramo (uma Series). `.map(normalizar_ramo)` aplica a função a **cada valor** da coluna e devolve a coluna nova, que é guardada de volta na mesma coluna. Note-se: `normalizar_ramo` vai **sem parênteses** porque se passa a função, não o resultado de a chamar.
- `pd.to_numeric(..., errors="coerce")` converte para número; o que não for convertível (por exemplo `"abc"`) vira `NaN` em vez de dar erro. É defesa contra dados sujos.
- `.fillna(0)` substitui valores em falta por 0 (aplicado aos montantes, para as somas funcionarem).

Nota: como o `fillna(0)` corre *antes* de `df.isna().sum()`, a contagem de valores em falta nos montantes nunca mostra problemas, mesmo que existissem. Para uma verificação de qualidade honesta, imprime-se primeiro e limpa-se depois. Nos dados atuais não há valores em falta, por isso o resultado não muda.

- `df.dtypes`: tipo de cada coluna; `df.isna().sum()`: quantos valores em falta por coluna; `df.duplicated().sum()`: quantas linhas repetidas; `.unique()`: valores distintos (confirma que o estado só tem Fechado/Pendente e que os 4 ramos estão bem escritos).

```
sinistro              str          sinistro              0
ramo                  str          ramo                  0
canal                 str          canal                 0
valor_reclamado     int64          ...                   0   (nenhuma coluna com falhas)
valor_regularizado  int64
dias_resolucao      int64          Duplicados: 0
estado                str
['Fechado', 'Pendente']   ['Automovel', 'Habitacao', 'Viagem', 'ResponsabilidadeCivil']
```

Nota técnica: em versões mais antigas do pandas, as colunas de texto aparecem como `object` em vez de `str`. É o mesmo conteúdo.

B.5 Colunas calculadas: valor_pendente e fora_prazo

```
    # 2. valor_pendente
    df["valor_pendente"] = df[COL_RECLAMADO].where(
        df[COL_ESTADO].astype(str).str.strip().eq("Pendente"), 0
    )

    # 3. fora_prazo
    df["fora_prazo"] = ~df.apply(
        lambda linha: cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS]), axis=1
    )
```

valor_pendente. Lê-se de dentro para fora:

- `df[COL_ESTADO].astype(str).str.strip()`: a coluna estado como texto, sem espaços.
- `.eq("Pendente")`: compara cada valor com `"Pendente"` e devolve uma coluna de `True/False` (equivale a `==`). A isto chama-se **máscara booleana**.
- `df[COL_RECLAMADO].where(máscara, 0)`: mantém o valor reclamado **onde a máscara é True** e põe 0 onde é False.
- Resultado: pendentes ficam com o valor reclamado, fechados ficam com 0. Escrever `df["valor_pendente"] = ...` cria a coluna nova no DataFrame.

fora_prazo.

- `df.apply(função, axis=1)`: chama a função **uma vez por linha** (`axis=1` = linha a linha; `axis=0` seria coluna a coluna). Em cada chamada, `linha` é uma Series com os valores dessa linha.
- `lambda linha: ...`: função anónima escrita numa linha. Equivale a `def f(linha): return cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS])`.
- O resultado é uma coluna de `True/False` a dizer se **cumpre** o prazo. O `~` à frente **inverte** (True↔False), porque fora de prazo é *não* cumprir. É a última linha do pseudocódigo: `fora_prazo ← NÃO cumpre_prazo`.

```
sinistro  ramo                   dias  limite  cumpre_prazo  fora_prazo
S001      Automovel                18      25  True          False
S002      Habitacao                32      25  False         True
S003      Viagem                    9      10  True          False
S004      ResponsabilidadeCivil    45      35  False         True
S005      Automovel                28      25  False         True
...       (no total: 7 de 12 fora de prazo)
```

B.6 Resumo por ramo (groupby)

```
    resumo = df.groupby(COL_RAMO).agg(
        n_sinistros=(COL_ID, "count"),
        valor_reclamado=(COL_RECLAMADO, "sum"),
        valor_regularizado=(COL_REGULARIZADO, "sum"),
        valor_pendente=("valor_pendente", "sum"),
        dias_medios=(COL_DIAS, "mean"),
        pct_fora_prazo=("fora_prazo", lambda s: s.mean() * 100),
    ).round(2)
    print(resumo)
```

- `df.groupby(COL_RAMO)` separa as linhas em 4 grupos, um por ramo (padrão *dividir–aplicar–combinar*).
- `.agg(nome=(coluna, operação))` (**agregação nomeada**) define, para cada grupo, uma coluna do resultado: o nome à esquerda é o da coluna nova, o par à direita diz de que coluna e que operação (`count`, `sum`, `mean`).
- `pct_fora_prazo`: os valores `True/False` contam como 1/0, por isso a média é a **proporção** de sinistros fora de prazo. Multiplicar por 100 dá a percentagem. A `lambda s` recebe a coluna `fora_prazo`

de cada grupo.

- `.round(2)` arredonda a 2 casas decimais.

```
ramo                 n  reclamado  regularizado  pendente  dias_medios  pct_fora_prazo
Automovel            3      11100          5400      5200        29.33           66.67
Habitacao            3      13800          7100      6200        24.00           33.33
ResponsabilidadeCivil  3      25800          7000     18200        46.00          100.00
Viagem               3       2830          1500      1200        10.00           33.33
```

Nota: o output do programa mostra as colunas do meio abreviadas com `...` porque o pandas encurta tabelas largas. Para as ver todas, acrescenta no início do `main()`: `pd.set_option("display.width", 200)` e `pd.set_option("display.max_columns", None)`.

B.7 Demonstração da classe

```
    # 5. Demonstração da classe
    linha = df.iloc[0]
    sinistro = Sinistro(
        linha[COL_ID], linha[COL_RAMO], linha[COL_ESTADO], linha[COL_RECLAMADO]
    )
    print(sinistro.id_sinistro, sinistro.valor_pendente())
```

- `df.iloc[0]`: a linha na **posição** 0 (a primeira), como Series. (`iloc` = posição; `loc` = etiqueta.)
- `linha[COL_ID]` etc. extraem os valores de que o construtor precisa. `Sinistro(...)` cria a **instância**, exigida pelo enunciado.
- `sinistro.id_sinistro` lê um atributo; `sinistro.valor_pendente()` chama o método (com parênteses).

```
S001 0
```

Nota: a primeira linha é o S001, que está **Fechado**, por isso o valor pendente é 0 e a demonstração não mostra o método a fazer nada. Com o primeiro pendente — `linha = df[df[COL_ESTADO] == "Pendente"].iloc[0]` — o output é `S002 6200`. Esta é uma das três correções sugeridas.

B.8 Exportação dos prioritários

```
    # 6. Exportação dos prioritários
    prioritarios = df[(df[COL_ESTADO].astype(str).str.strip().eq("Pendente")) | df["fora_prazo"]]
    prioritarios = prioritarios.sort_values(
        by=["valor_pendente", COL_DIAS], ascending=[False, False]
    )
    prioritarios.to_csv(base_dir / "sinistros_prioritarios.csv", index=False)
```

- `df[máscara]`: **filtragem** — devolve só as linhas onde a máscara é True.
- A máscara combina duas condições com `|` (**OU** lógico entre colunas; `&` seria E). Os **parênteses** são obrigatórios porque `|` tem prioridade sobre `==` e `.eq()`. Não se usa `or` porque só funciona com valores únicos, não com colunas.
- Resultado: pendentes *ou* fora de prazo → 7 sinistros (5 pendentes, todos fora de prazo, e 2 fechados que foram resolvidos tarde).
- `.sort_values(by=[...], ascending=[False, False])`: ordena por valor pendente (maior primeiro) e, em caso de empate, por dias de resolução (maior primeiro). Cada critério tem a sua direção em `ascending`.
- `.to_csv(caminho, index=False)`: grava o CSV. O `index=False` evita escrever a coluna 0,1,2... que o pandas cria por defeito (o 'índice técnico' que o enunciado proíbe).

```
sinistro,ramo,canal,valor_reclamado,...,dias_resolucao,estado,valor_pendente,fora_prazo
S012,ResponsabilidadeCivil,Corretor,9800,0,55,Pendente,9800,True
S004,ResponsabilidadeCivil,Agente,8400,0,45,Pendente,8400,True
S002,Habitacao,Digital,6200,0,32,Pendente,6200,True
S009,Automovel,Corretor,5200,0,42,Pendente,5200,True
S007,Viagem,Digital,1200,0,14,Pendente,1200,True
S008,ResponsabilidadeCivil,Agente,7600,7000,38,Fechado,0,True      <- fechados vêm no fim
S005,Automovel,Digital,2100,1900,28,Fechado,0,True
```

B.9 Apoio às questões de análise e arranque

```
    print(df[COL_CANAL].value_counts(normalize=True))
    print(prioritarios[COL_CANAL].value_counts(normalize=True))
    print(pd.crosstab(prioritarios[COL_CANAL], prioritarios[COL_RAMO]))


if __name__ == "__main__":
    main()
```

- `.value_counts(normalize=True)`: conta quantas vezes aparece cada canal e converte em **proporção** (soma 1). Comparar a proporção de canais em *todos* os sinistros com a dos *prioritários* responde à pergunta 'concentram-se num canal?'.
- `pd.crosstab(a, b)`: tabela de contagens cruzando duas variáveis (canal nas linhas, ramo nas colunas).
- `if __name__ == "__main__":`: `__name__` vale `"__main__"` quando o ficheiro é **executado diretamente** e o nome do módulo quando é **importado**. Assim `main()` só corre no primeiro caso — mesmo padrão que impede o `funcoes_auxiliares` de executar código ao ser importado.

```
canal (todos)      Agente 0.333  Corretor 0.333  Digital 0.333
canal (prioritários)  Digital 0.429  Corretor 0.286  Agente 0.286

ramo      Automovel  Habitacao  ResponsabilidadeCivil  Viagem
Agente            0          0                      2       0
Corretor          1          0                      1       0
Digital           1          1                      0       1
```

Parte C — Para a apresentação

Decisões de código a destacar

- **Limite inclusivo (`<=`)**: resolver no último dia ainda cumpre o prazo.
- **Sem informação = dentro do prazo**: evita falsos alarmes; a limitação é que ramos mal escritos passam despercebidos (por isso a verificação de qualidade e a normalização).
- **Regras separadas do fluxo** (módulo auxiliar): a lógica de negócio fica isolada, reutilizável e testável.
- **Conversão defensiva dos números** (`errors="coerce"`): dados sujos não rebentam o programa.
- **Percentagem via média de booleanos**: truque compacto do pandas, mas fácil de explicar.

Perguntas que podem fazer — respostas curtas

Porquê `apply` e não um ciclo `for`?

Mais curto e legível; o pandas gere a iteração. Para tabelas enormes seria mais rápido vetorizar, mas o enunciado pede usar as funções definidas.

O que acontece com um ramo novo?

Fica com o nome original, `limite_regularizacao` devolve `None` e `cumpre_prazo` devolve `True`: não é assinalado como atrasado. É uma limitação a referir.

Diferença entre `.map` e `.apply`?

`.map` aplica uma função a cada **valor** de uma coluna; `.apply(axis=1)` aplica-a a cada **linha** (acesso a várias colunas).

Porquê `index=False`?

Sem isso o CSV teria uma coluna extra com 0,1,2..., o 'índice técnico' que o enunciado proíbe.

Para que serve a classe se já há `valor_pendente` no DataFrame?

Mostra a mesma regra orientada a objetos, aplicada a um sinistro individual; cumpre o requisito de classe com atributos, método e instância.

O que faz o `~`?

Inverte uma coluna booleana: transforma 'cumpre o prazo' em 'fora de prazo'.

As 3 correções sugeridas

- **Demonstração da classe com um pendente** (B.7): `df[df[COL_ESTADO] == "Pendente"].iloc[0]`.
- **Verificar a qualidade antes do `fillna(0)`** (B.4): imprimir `isna().sum()` antes de limpar.
- **Não repetir a expressão do estado** (B.5 e B.8): guardar `pendente = df[COL_ESTADO].astype(str).str.strip().eq("Pendente")` e reutilizar.